DATA STRUCTURES HOME WORK 2

1)Define an integer array. Declare an integer pointer & initialize it to point to first element of array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

CODE:

```c
#include<stdio.h>
int main()
{
    int n;
    printf("\nEnter the no of elements:");
    scanf("%d",&n);
    int arr[n];
    int *ptr;
    ptr=arr;
    for(int i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\nArray elments:");
    for(int i=0;i<n;i++)
    {
        printf("%d\t",*ptr);
        ptr++;
    }
    int m;
    printf("\nEnter the position to be modified:");
    scanf("%d",&m);
    int num;
    printf("\nEnter the new element:");
    scanf("%d",&num);
    ptr=arr;
    m--;
    *(ptr+m)=num;
    printf("\nArray elements after Modifying:");
    for(int i=0;i<n;i++)
    {
        printf("%d\t",*ptr);
        ptr++;
    }
}
```

OUTPUT:

Enter the no of elements:3

1

2

3

Array elments:1 2 3

Enter the position to be modified:2

Enter the new element:5

Array elements after Modifying:1 5 3

---

2)Calculate string length using pointers.

CODE:

```c
#include<stdio.h>
int main()
{
    char str[25];
    printf("Enter a string:");
    scanf("%s",str);
    char *ptr;
    ptr=str;
    int count=0;
    while(*ptr!='\0')
    {
        count++;
        ptr++;
    }
    printf("Length of the string:%d",count);
}
```

OUTPUT:

Enter a string:Visalini

Length of the string:8

---

3) Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

CODE:

```c
#include<stdio.h>
void sp(int*,int*,int*,int*);
int main()
{
```

```
    int a,b,ad,mu;
    printf("Enter first number:");
    scanf("%d",&a);
    printf("Enter second number:");
    scanf("%d",&b);
    sp(&a,&b,&ad,&mu);
    printf("Addtion=%d",ad);
    printf("\nMultiplication:%d",mu);
}
void sp(int *p1,int *p2,int *s,int *p)
{
    *s=*p1+*p2;
    *p=(*p1)*(*p2);
}
```

OUTPUT:

Enter first number:2

Enter second number:3

Addtion=5

Multiplication:6

---

4) Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
    int rno;
    char name[25];
};
int main()
{
    int n;
    printf("Enter the no of students:");
    scanf("%d",&n);
    struct student *ptr;
    ptr = (struct student *)malloc(n * sizeof(struct student));
    for(int i=0;i<n;i++)
    {
        printf("\nEnter Rollno:");
        scanf("%d",&(ptr+i)->rno);
```

```
        printf("\nEnter Name:");
        scanf("%s",(ptr+i)->name);
    }
    for(int i=0;i<n;i++)
    {
        printf("\nRollno:%d",(ptr+i)->rno);
        printf("\nName:%s",(ptr+i)->name);
    }
    free(ptr);
}
```

OUTPUT:

Enter the no of students:2

Enter Rollno:7

Enter Name:Dhoni

Enter Rollno:18

Enter Name:Virat

Rollno:7

Name:Dhoni

Rollno:18

Name:Virat

---

5) Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

CODE:

```
#include<stdio.h>
void add(int,int);
void sub(int,int);
int main()
{
    void (*fp) (int,int);
    fp=add;
    (*fp)(5,3);
    fp=sub;
    (*fp)(5,3);
}
void add(int a,int b)
{
```

```
    printf("Addition:%d",a+b);
}
void sub(int a,int b)
{
    printf("\nSubtraction:%d",a-b);
}
```

OUTPUT:

Addition:8

Subtraction:2

---

6) Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

CODE:

```
#include<stdio.h>
int main()
{
    int a=10;
    int*p;
    int**p1;
    p=&a;
    p1=&p;
    printf("Value in a:%d",a);
    printf("\nValue in a using p:%d",*p);
    printf("\nValue in a using p1:%d",**p1);
    printf("\nAddress of a:%d",&a);
    printf("\nAddress of a using p:%d",p);
    printf("\nAddress of p:%d",&p);
    printf("\nAddress of p using p1:%d",p1);
    printf("\nAddress of p1:%d",&p1);
}
```

OUTPUT:

Value in a:10

Value in a using p:10

Value in a using p1:10

Address of a:6422300

Address of a using p:6422300

Address of p:6422296

Address of p using p1:6422296

Address of p1:6422292

---

7) Sort an array of floating point numbers using selection sort.

CODE:

```c
#include<stdio.h>
int smallest(float[],int,int);
void sel(float[],int);
int main()
{
    int n;
    printf("Enter the no of elements:");
    scanf("%d",&n);
    float num[n];
    printf("Enter Numbers:");
    for(int i=0;i<n;i++)
    {
        scanf("%f",&num[i]);
    }
    sel(num,n);
    printf("Numbers after sorting:");
    for(int i=0;i<n;i++)
    {
        printf("\n%.2f",num[i]);
    }
    return 0;
}
int smallest(float num[],int n,int k)
{
    float small=num[k];
    int pos=k;
    for(int i=k+1;i<n;i++)
    {
        if(num[i]<small)
        {
           small=num[i];
           pos=i;
        }
    }
    return pos;
}
void sel(float num[],int n)
{
    int k,pos;
    float temp;
    for(k=0;k<n;k++)
```

```
    {
        pos=smallest(num,n,k);
        temp=num[k];
        num[k]=num[pos];
        num[pos]=temp;
    }
}
```

OUTPUT:

Enter the no of elements:4

Enter Numbers:

2.4

6.4

3.7

8.43

Numbers after sorting:

2.40

3.70

6.40

8.43

---

8) Define a structure `Student` with `id`, `name` and `grade`, create an array of `Student` structures and sort them based on different fields.

CODE:

```
#include<stdio.h>
#include<string.h>
struct student
{
    int id;
    char name[25];
    char grade;
};
int main()
{
    int n,choice;
    printf("\nEnter the no of students:");
    scanf("%d",&n);
```

```c
    struct student s[n],temp;
    for(int i=0;i<n;i++)
    {
        printf("\nEnter ID:");
        scanf("%d",&s[i].id);
        printf("\nEnter Name:");
        scanf("%s",s[i].name);
        printf("\nEnter Grade:");
        scanf(" %c",&s[i].grade);
    }
    printf("\nSort by:");
    printf("\n1. ID");
    printf("\n2. Name");
    printf("\n3. Grade");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    for(int i = 0; i < n - 1; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            int swap = 0;

            if(choice == 1 && s[i].id > s[j].id)
                swap = 1;

            if(choice == 2 && strcmp(s[i].name, s[j].name) > 0)
                swap = 1;

            if(choice == 3 && s[i].grade > s[j].grade)
                swap = 1;

            if(swap)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nSorted Students:\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t%s\t%c\n",
               s[i].id, s[i].name, s[i].grade);
```

```
    }

    return 0;
}
```

OUTPUT:

Enter the no of students:2

Enter ID:92

Enter Name:visalini

Enter Grade:A

Enter ID:52

Enter Name:yoga

Enter Grade:B

Sort by:

1. ID

2. Name

3. Grade

Enter choice: 2

Sorted Students:

92 visalini A

52 yoga B

---

9)Sort a set of negative numbers using radix sort.

CODE:

```
#include <stdio.h>

int largest(int arr[], int n);
void radix_sort(int arr[], int n);

int main()
{
    int arr[10], n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);
```

```c
    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    int min=arr[0];
    for(i = 0; i < n; i++)
    {
        if(arr[i]<min)
            min=arr[i];
    }

    for(i = 0; i < n; i++)
    {
        arr[i]=arr[i]-min;
    }

    radix_sort(arr, n);

    for(i = 0; i < n; i++)
    {
        arr[i]=arr[i]+min;
    }

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t", arr[i]);
    }

    return 0;
}

int largest(int arr[], int n)
{
    int i, largest = arr[0];

    for(i = 1; i < n; i++)
    {
        if(arr[i] > largest)
        {
            largest = arr[i];
        }
    }

    return largest;
```

```
}

void radix_sort(int arr[], int n)
{
    int bucket[10][10], bucketCount[10];
    int i, j, k;
    int remainder, NOP = 0, divisor = 1, large, pass;

    large = largest(arr, n);

    while(large > 0)
    {
        NOP++;
        large = large / 10;
    }

    for(pass = 0; pass < NOP; pass++)
    {
        for(i = 0; i < 10; i++)
            bucketCount[i] = 0;

        for(i = 0; i < n; i++)
        {
            remainder = (arr[i] / divisor) % 10;
            bucket[remainder][bucketCount[remainder]] = arr[i];
            bucketCount[remainder]++;
        }

        i = 0;
        for(k = 0; k < 10; k++)
        {
            for(j = 0; j < bucketCount[k]; j++)
            {
                arr[i] = bucket[k][j];
                i++;
            }
        }

        divisor = divisor * 10;
    }
}
```

OUTPUT:

Enter the number of elements: 4

Enter the elements:

0

-27

-1

18

Sorted array:

-27 -1 0 18

---

10)Write a C program to copy one array to another using pointers.

CODE:

```c
#include<stdio.h>
int main()
{
    int n;
    printf("\nEnter the no of elements:");
    scanf("%d",&n);
    int arr[n],cp[n];
    int *p1=&arr[n-1];
    int *p2=cp;
    for(int i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    for(int i=0;i<n;i++)
    {
        *(p2+i)=*p1;
        p1--;
    }
    printf("\nReversed Array:");
    for(int i=0;i<n;i++)
    {
        printf("\n%d",cp[i]);
    }
}
```

OUTPUT:

Enter the no of elements:3

1

2

3

Reversed Array:

3

2

1
